

LEGO-RL: Harness-Native Reinforcement Learning for Coding Agents

Yiming Du^{1,2,*} Yuxin Jiang^{1,*} Tao Yuan^{1,*} Jianbo Dai¹ Shaowei Wang¹ Jierun Chen¹
 Chaofan Tao¹ Xianzhi Yu¹ Lifeng Shang¹ Kam-Fai Wong² Xiaohui Li^{1,†} Haoli Bai^{1,†}

¹Huawei Technologies Co., Ltd ²The Chinese University of Hong Kong

 <https://github.com/LegoX/Lego-RL>  <https://huggingface.co/LegoX/Lego-RL>

Abstract

Reinforcement learning for coding agents increasingly relies on long-running agent harnesses to manage tool integration, repository contexts, and execution feedback. However, the native execution environments of these harnesses are inherently misaligned with policy-gradient training: environmental crashes and reward hacking corrupt outcome signals, while train–inference discrepancies decouple rollout behavior from policy updates. To address this, we present LEGO-RL, a framework that bridges native coding-agent harnesses with scalable policy-gradient optimization without modifying their internal control flow. LEGO-RL is built upon three pillars: **(1) faithful optimization** via in-process LLM proxying that captures raw generation streams for token-level alignment and robust trainer-side log-probability recomputation, even under harness-side compaction or re-serialization; **(2) reliable execution** via scalable sandbox orchestration featuring image caching and stage-wise defenses to mitigate reward hacking; and **(3) observable training** through an integrated plugin that automates validation and monitoring, paired with a Live UI for granular trajectory diagnostics. We evaluate LEGO-RL by training the sparse MoE model Qwen3.5-35B-A3B with GSPO across three native coding-agent harnesses. LEGO-RL improves Qwen3.5-35B-A3B across OpenHands SDK (64.0% → 70.4%), Claude Code (62.4% → 68.2%), and OpenCode (57.2% → 66.6%) on SWE-bench Verified, while maintaining a rollout–training probability correlation above 0.99.

1 Introduction

Training coding agents with reinforcement learning requires optimizing long-horizon behavior over software repositories, tools, and execution environments (Wei et al., 2025; Golubev et al., 2025). Rather than learning from isolated responses, reinforcement learning updates the policy from complete trajectories and their verifier rewards. A single rollout may involve repeated model calls, repository inspection, tool calling, code modification, dependency installation, and test execution before producing a sparse executable reward (Jimenez et al., 2024; Pan et al., 2024). Because these trajectories are generated by native agent harnesses that manage prompts, tools, context, and execution state (Yang et al., 2024; Wang et al., 2025b), the optimization target is the policy behavior induced by the original agent control flow.

Integrating native coding-agent harnesses such as SWE-agent (Yang et al., 2024), Claude Code (Anthropic, 2025), OpenHands SDK (Wang et al., 2025b), and OpenCode (SST, 2025) into policy-gradient

*Co-first authors. [†]Corresponding authors: {lixiaohui33, baihaoli}@huawei.com

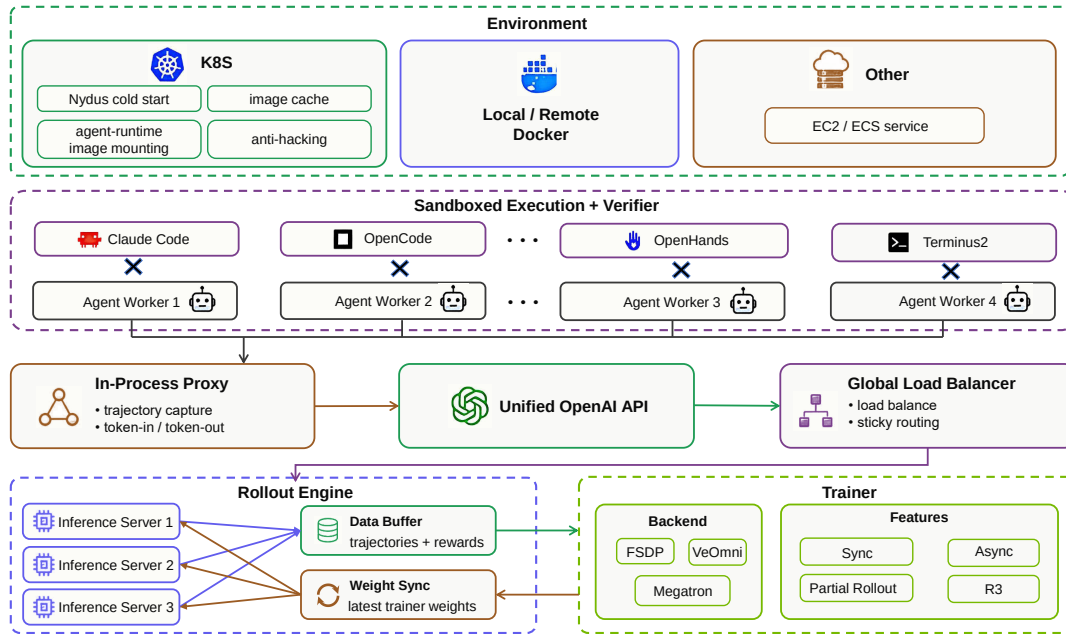


Figure 1: Overview of the LEGO-RL training infrastructure.

training remains challenging. Harness-side prompt construction, context compaction, and history rewriting can cause the reconstructed trajectory to differ from the exact token sequence sampled during rollout, preventing faithful trainer-side probability recomputation (Xu et al., 2026). Sparse mixture-of-experts models introduce an additional mismatch when rollout-time expert routing is not reproduced during training (Ma et al., 2025). At the execution layer, sandbox failures, dependency errors, verifier misconfiguration, timeouts, and reward hacking can discard costly trajectories or corrupt reward signals, yet are difficult to localize in asynchronous pipelines where failures propagate across stages and diagnostic signals are fragmented across workers. General RL frameworks often require coding agents to be adapted to framework-defined rollout interfaces (Sheng et al., 2025; Cao et al., 2025); for example, OpenHands-based agents may require overridden initialization, replacement of the default tool set with task-specific tools, and custom termination logic for reward extraction (Cao et al., 2025; Sutawika et al., 2026). Harness-native approaches preserve the existing agent workflow (Yu et al., 2026), but scalable execution, reward-integrity protection, and trajectory-level diagnosis remain insufficiently addressed.

We present LEGO-RL, a framework that connects existing coding-agent harnesses to scalable policy-gradient optimization without modifying their agent workflows. Built on verl (Sheng et al., 2025) and Harbor (The Harbor Team, 2026), it integrates reinforcement learning and sandboxed execution around the existing harness while preserving its model APIs, tool interfaces, prompt construction, and context-management policies. Supporting a new harness requires only a lightweight adapter that launches the agent, connects it to the inference service, and returns the resulting interaction data; the remaining training pipeline is shared across agents. The framework is organized around three core pillars:

- **Faithful optimization.** An in-process LLM proxy captures exact rollout tokens and generation metadata, enabling faithful trainer-side log-probability recomputation despite harness-side history rewriting. For mixture-of-experts models, rollout-time routing decisions are replayed during training.
- **Reliable execution.** Scalable sandbox orchestration, image caching, and stage-wise defenses protect re-

Framework	Harness-native fidelity			Execution & reward			Observability	
	Black-box harness	Token-in/Token-out	History alignment	R3	Fully async	Sandbox execution	Reward-hack defense	Training observability
verl (2025)	–	✓	–	✓	✓	△	–	△
slime (2025)	✓	✓	✓	✓	✓	△	△	–
MOLT (2026)	△	✓	–	✓	✓	–	–	–
SkyRL-Agent (2025)	△	△	△	✓	✓	✓	–	–
AReal (2025)	✓	△	–	–	✓	–	–	–
Agent Lightning (2025)	△	✓	–	–	△	–	–	△
Polar (2026)	✓	✓	✓	–	✓	✓	–	–
rLLM (2026)	✓	✓	–	✓	✓	✓	–	✓
OpenForgeRL (2026)	✓	△	–	–	✓	✓	–	–
ALE (ROLL/ROCK) (2025a)	–	–	–	–	✓	✓	△	–
LEGO-RL	✓	✓	✓	✓	✓	✓	✓	✓

Table 1: Comparison of representative agentic RL frameworks. ✓: supported; △: partial/conditional support; –: not reported. R3: rollout routing replay. Observability denotes monitoring training runs and diagnosing execution- or trajectory-level failures.

ward integrity, while termination-aware filtering and recovery prevent failed trajectories from disrupting asynchronous training.

- **Observable training.** Automated run validation, real-time monitoring, and trajectory-level diagnosis expose failures across agents, environments, verifiers, and optimization.

We evaluate LEGO-RL across OpenHands SDK, Claude Code, and OpenCode on a sparse mixture-of-experts policy. Across the three harnesses, LEGO-RL yields large gains in resolve rate on SWE-bench Verified: 64.0% → 70.4% on OpenHands SDK, 62.4% → 68.2% on Claude Code and 57.2% → 66.6% on OpenCode, while maintaining the rollout–training probability correlation above 0.99. To facilitate reproducibility, we open-source the complete framework, trained models, and datasets for the community.

2 Related Work

Agentic RL training frameworks. Early LLM-RL infrastructure such as verl (Sheng et al., 2025) placed rollout generation within the trainer-managed pipeline, with the trainer directly controlling model generation and trajectory construction. Recent frameworks can be broadly categorized by where the agent rollout is implemented. slime (THUDM, 2025), MOLT (NVIDIA NeMo, 2026), SkyRL (Cao et al., 2025), and AReal (Fu et al., 2025) retain the rollout within the RL framework and provide customizable or asynchronous agentic rollout support, but existing agents must be adapted to their interaction and environment abstractions. ALE co-designs the ROLL trainer, ROCK sandbox manager, and iFlow CLI agent, achieving consistency through control of the full stack (Wang et al., 2025a). In contrast, Polar, rLLM, and OpenForgeRL preserve existing agent harnesses and observe their interactions at the model API (Xu et al., 2026; Berkeley Sky Computing Lab, 2026; Yu et al., 2026), while Agent Lightning uses SDK callbacks (Luo et al., 2025). LEGO-RL follows the model-API approach and integrates it with policy-gradient training, sandboxed execution, and executable verification. Table 1 compares existing RL training frameworks across harness-native fidelity, execution and reward integrity, training operations, and held-out SWE-bench evaluation. Although individual capabilities are supported by several existing frameworks, LEGO-RL brings them together in a single harness-native policy-gradient training framework.

Coding-agent benchmarks, harnesses, and training tasks. SWE-bench introduced repository-level issue resolution with executable validation (Jimenez et al., 2024), later extended to harder tasks in SWE-bench Pro (Deng et al., 2025), multilingual repositories in Multi-SWE-bench (Zan et al., 2025), and continuously refreshed tasks in SWE-rebench (Badertdinov et al., 2025). Unlike single-step LLM-RL, these tasks require many interleaved model calls, tool actions, and environment transitions before receiving a sparse verifier reward (Zhang et al., 2025; Xi et al., 2025). Harness design is therefore part of the optimization problem, since repository navigation, editing, and execution interfaces directly affect agent behavior (Yang et al., 2024). Prior work on SWE-RL either omits executable interaction (Wei et al., 2025) or trains within a framework-controlled agent loop (Agentica Team and Together AI, 2025; Golubev et al., 2025). Existing task collections provide executable environments through different construction pipelines, including SWE-Gym, R2E-Gym, SWE-Smith, OpenSWE, and SWE-Universe (Pan et al., 2024; Jain et al., 2025; Yang et al., 2025; Fu et al., 2026; Chen et al., 2026). Rather than introducing another collection, we study task selection for agentic RL based on a scalable sandbox environment, reliable verifier, and policy-relative difficulty.

3 The LEGO-RL Framework

3.1 Overview

LEGO-RL consists of a harness-native training infrastructure embedded within a broader closed-loop operational workflow. We first formalize the problem setup and its faithfulness requirements, then describe the training infrastructure in §3.2, followed by the closed-loop operational workflow in §3.3.

3.1.1 Problem Setup and Objective

Harness-native rollouts. A task instance $x = (q_x, R_x, V_x)$, drawn from a task pool $\mathcal{D}$, pairs a problem statement q_x and an initialized repository environment R_x with a task-specific executable verifier V_x . We treat the native coding-agent harness $\mathcal{H}$ as part of the environment and optimize only the policy π_θ it calls. At turn $t = 1, \dots, T$, the harness maps the current interaction and repository state s_t to a context $c_t = \mathcal{H}(s_t)$, the policy generates an assistant token span $a_t \sim \pi_\theta(\cdot \mid c_t)$, and the harness executes the requested tool actions, yielding s_{t+1} . A rollout is the sequence of prompt–response pairs actually exchanged at the model API, $\tau = ((c_1, a_1), \dots, (c_T, a_T))$, and the verifier assigns a single terminal binary reward $r(x, \tau) = V_x(s_{T+1}) \in \{0, 1\}$. Only policy-generated tokens are trained on: let $\mathcal{M}(\tau)$ denote the set of token positions corresponding to policy-generated response tokens. The trajectory log-likelihood is $\log \pi_\theta(\tau) = \sum_{(t,j) \in \mathcal{M}(\tau)} \log \pi_\theta(a_{t,j} \mid c_t, a_{t,<j})$, where each turn is conditioned on the harness-supplied context c_t rather than on the raw history—a distinction that §3.2.2 shows to be essential.

Objective. The expected verifier reward $J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \tau \sim \mathcal{H}(\pi_\theta)} [r(x, \tau)]$ is maximized with group-relative advantage estimation: each task receives a group of G trajectories with rewards $r_i = r(x, \tau_i)$ and $\hat{A}_i = (r_i - \bar{r}) / (\text{std}(r_{1:G}) + \delta)$, where $\delta = 10^{-6}$ keeps the estimator defined when the group’s reward variance is zero. Our main experiments use the GSPO (Zheng et al., 2025) sequence-level surrogate

$$\mathcal{J}_{\text{GSPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G w(\tau_i) \min \left(\sigma_i(\theta) \hat{A}_i, \text{clip}(\sigma_i(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \hat{A}_i \right) \right], \quad (1)$$

$$\sigma_i(\theta) = \left(\frac{\pi_\theta(\tau_i)}{\pi_{\theta_{k'}}(\tau_i)} \right)^{1/|\mathcal{M}(\tau_i)|},$$

where $\theta_{k'}$ is the policy version that generated the group and $w(\tau_i) \in \{0, 1\}$ filters invalid trajectories caused by infrastructure or execution failures, preventing them from corrupting the training signal §3.2.3. The

asymmetric bounds $\epsilon_{\text{low}} < \epsilon_{\text{high}}$ (Table 10) admit more upward than downward movement in the sequence-level ratio; token-level PPO (Schulman et al., 2017) and GRPO (Shao et al., 2024) objectives are supported by replacing σ_i with the per-token ratio. Two consequences shape the rest of the system: a group with equal rewards gives $\hat{A}_i \equiv 0$ and contributes no gradient, making policy-relative task difficulty and pool composition a first-order concern, and r is produced by executing code, so the learning signal is only as trustworthy as the sandbox and the verifier.

Faithfulness requirement. The trajectory log-likelihood is well defined only if the trainer sees the exact contexts, tokens, and mask of the rollout. Because a real harness may compact, re-serialize, or rewrite its interaction history between turns, the recorded transcript need not decode and re-encode back to the sampled token sequence, so τ and $\mathcal{M}(\tau)$ must be captured at the model-serving boundary rather than reconstructed. Let $\ell_{i,(t,j)}^{\text{roll}}$ be the log-probability recorded at generation time and $\ell_{i,(t,j)}^{\text{train}}(\theta)$ the value the trainer recomputes. Faithful optimization requires $\ell_{i,(t,j)}^{\text{train}}(\theta_{k'}) \approx \ell_{i,(t,j)}^{\text{roll}}$ for every $(t, j) \in \mathcal{M}(\tau_i)$: agreement on the *same* weights $\theta_{k'}$, up to numerical tolerance. That holds only if token IDs, response masks, and policy weights match, and, for sparse mixture-of-experts policies, only if training reuses the expert-routing decisions of the behavior policy. Under fully asynchronous training the trainer version θ_k may lead the behavior version $\theta_{k'}$; this explicitly bounded staleness is off-policyness corrected by Eq. (1), not a capture error, whereas any violation of the agreement above is.

3.2 Training Infrastructure

Figure 1 depicts the overview of the LEGO-RL training infrastructure. An unmodified coding-agent harness runs inside a per-trial sandbox on Kubernetes, Docker, or a cloud container service. Every model call it issues passes through the in-process proxy, which records token IDs, log-probabilities, response masks, and expert-routing decisions, and is then routed to an inference server with sticky routing. Completed trajectories and verifier rewards enter the data buffer; the trainer consumes them and pushes updated weights back to the inference servers. Only the sandbox layer is harness-specific.

3.2.1 Sandbox Execution Environment

Every trial runs in a fresh, isolated sandbox, where the task environment is constructed, the agent interacts with the repository, and the final state is evaluated through executable verification. At scale, the sandbox must support efficient environment preparation, reliable concurrent execution, and trustworthy verifier rewards. We therefore organize its design around environment setup and isolation, execution reliability, and reward integrity.

Environment setup and isolation. LEGO-RL abstracts sandbox execution behind a common interface supporting multiple backends, such as Docker and Kubernetes. To reduce startup overhead, a Nydus lazy-pull snapshotter backed by shared storage streams image chunks on demand, avoiding full image replication across nodes. Components absent from task images, including the pinned agent runtime and grading toolchain, are mounted read-only rather than reinstalled for each trial. Tasks without prebuilt images use *inline image build*, with pinned dependencies and fail-fast setup separating environment failures from policy failures. Per-pod CPU and memory limits isolate excessive resource use, while temporary package-extraction writes are redirected to an in-memory `emptyDir` to reduce local-storage contention under concurrency.

Execution performance and scheduling. Agent execution dominates trial duration, while sandbox setup and verification contribute relatively little on average but exhibit substantial tail latency. This motivates

stage-specific rather than trial-wide timeouts and asynchronous scheduling, which prevents pathological trials from delaying an entire synchronized rollout batch. §4.6 reports the measured stage-wise decomposition that supports these choices.

Reward integrity. Executable verification provides a trustworthy reward signal only when positive rewards correspond to genuine task completion. We observed both agent-side shortcuts that expose grading information and environment-side failures that make rewards independent of agent behavior. LEGO-RL therefore enforces reward-integrity defenses within the sandbox: network restrictions are controlled by a privileged sidecar that the agent cannot modify, repository history is hidden during execution and restored when required for verification, and test dependencies are packaged into the task image to eliminate reliance on external network state. The observed failure modes, corresponding defenses, and audit results are summarized in Table 6 of Appendix A.

3.2.2 In-Process Proxy

The in-process proxy connects unmodified coding-agent harnesses to the optimizer at the provider API boundary. Co-located with the rollout engine, it supports both OpenAI-compatible and Anthropic APIs and captures token IDs, log-probabilities, response masks, and generation metadata directly from serving sessions, preserving policy-generated tokens at generation time rather than reconstructing them from the final trajectory.

Alignment under history rewriting. Because a harness may re-serialize, compact, or drop interaction history between model calls, LEGO-RL aligns successive contexts at message granularity before assembling a training trajectory. System, user, and tool-result messages must match exactly, while tool calls are associated through their stable identifiers and function names so that argument reserialization does not alter the captured policy tokens. Matched policy-generated spans retain their original token IDs, log-probabilities, and response masks; rewritten or harness-authored content is treated only as conditioning context and is not added to $\mathcal{M}(\tau)$. Calls belonging to sub-agents are isolated from the parent capture session to prevent their tokens from entering the training trajectory. If the history cannot be aligned reliably, the affected content is excluded rather than reconstructed from the modified transcript. For sparse mixture-of-experts policies, the proxy additionally records rollout-time routing decisions and replays them during training through R3 (Ma et al., 2025), ensuring that trainer-side probability computation follows the same expert routing as the behavior policy. §4.4 evaluates alignment under history rewriting, rollout-training agreement, and routing replay.

3.2.3 Rollout and Training

Captured trials reach the optimizer through two stages that this section covers together, because they are tuned as one system: a scheduler that keeps a fixed pool of inference slots productive under a heavy right tail, and a verl-based trainer that consumes the filtered batches.

Rollout. Agentic coding rollouts vary substantially in duration and exhibit a heavy right tail, which can reduce inference utilization under synchronized generation (Fu et al., 2025). LEGO-RL therefore uses fully asynchronous rollout generation, decoupling trajectory generation from optimization and starting new rollouts as soon as previous ones finish. Configurable limits bound exceptionally long sessions, while partial trajectories spanning weight synchronization are recovered rather than discarded. Before optimization, trajectories are handled according to their termination status: execution failures are masked from training,

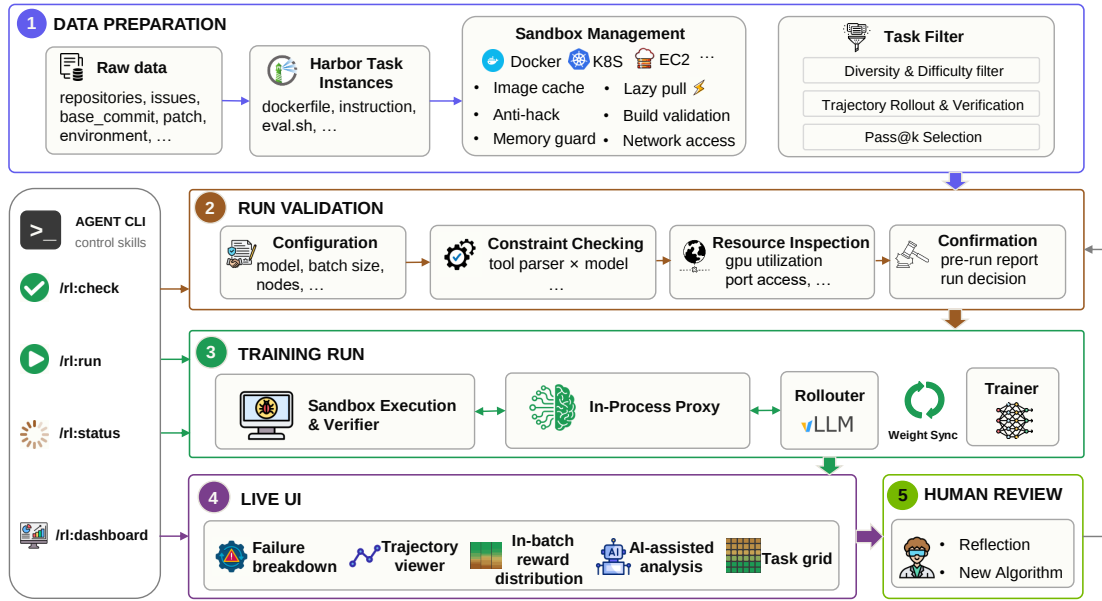


Figure 2: Closed-loop operational workflow of LEGO-RL. The five stages cover data preparation, run validation, training, live observability, and human review. Stage (3) corresponds to the training infrastructure shown in Figure 1, while the agent plugin acts as the control plane.

whereas valid but incomplete policy trajectories are retained. Masked trajectories are kept for batch consistency but assigned zero optimization weight, ensuring that execution failures do not contribute to group-relative policy updates.

Training. The training layer builds on verl (Sheng et al., 2025), inheriting support for PPO (Schulman et al., 2017), GRPO (Shao et al., 2024), GSPO (Zheng et al., 2025), vLLM (Kwon et al., 2023) serving, and rollout scheduling. LEGO-RL adds the integration and backend support required for harness-native agentic RL, with **VeOmni**, **FSDP**, and **Megatron** (Shoeybi et al., 2019) supported as training backends. Both synchronous and fully asynchronous training are supported, with bounded policy staleness in the asynchronous setting (§4.6).

3.3 Closed-Loop RL Workflow

Beyond the training infrastructure, LEGO-RL organizes the experiment lifecycle into five stages: **Data Preparation**, **Run Validation**, **Training Run**, **Live UI**, and **Human Review** (Figure 2). Stage (1) converts task descriptions and repository snapshots into executable task instances, builds the corresponding sandbox environments, and selects training candidates based on repository diversity and policy-relative difficulty; Appendix J documents the three representations a task passes through. Stage (2) validates experiment configurations, cross-parameter constraints, and resource availability before execution. Stage (3) runs the sandbox-to-optimizer training loop shown in Figure 1. Stage (4) presents optimization metrics, rollout statistics, termination causes, and trajectory-level evidence through the Live UI. In stage (5), researchers use this evidence to diagnose failures, analyze learned behaviors, formulate new hypotheses, and guide subsequent experiments and system refinement, thereby closing the loop. The agent plugin provides reusable skills that automate and coordinate stages (2)–(4): *Run Validation*, *Training Run*, and *Live Observability*. It spans these stages rather than constituting an additional workflow stage. The following subsections describe

the agent plugin and Live UI in detail.

3.3.1 Agent Plugin

The agent plugin exposes operational capabilities through reusable skills spanning run validation, training execution, and live monitoring. Before an experiment starts, these skills resolve the experiment configuration, validate cross-parameter constraints and resource availability, and summarize the results for operator confirmation. During execution, they coordinate existing scripts and services to launch experiments, track progress, diagnose common failures, and access relevant monitoring views. Rather than duplicating training logic, the plugin composes existing system capabilities into a consistent workflow for coding agents and human operators. The same skills support training, standalone evaluation, and batch inference, reducing manual setup and configuration inconsistencies while preserving direct access to the underlying commands.

3.3.2 Live UI

The Live UI extends standard experiment tracking with cross-stage diagnostics for agentic RL by linking verifier outcomes to termination states, task instances, agent trajectories, tool usage, and rollout–training consistency. It provides three complementary views: termination-reason distributions and stage-level timing for identifying the source of reward variation; per-instance task grids and trajectory views for relating training metrics to task-level solve-rate changes, interaction patterns, and tool use; and consistency and in-batch distribution views for assessing rollout–training alignment and whether sampled groups retain sufficient reward variation for group-relative optimization. Together, these views connect changes in training metrics to their underlying task- and trajectory-level evidence. §4.5 evaluates this capability on representative failure cases observed during training, while Appendix H provides the full panels, interface examples, and supporting mechanisms for training-state tracking, verifier re-evaluation, and trajectory export.

4 Experiments

We evaluate LEGO-RL along five dimensions. §4.1 describes the experimental setup and §4.2 reports the main results under three native coding-agent harnesses. §4.3, §4.4, and §4.5 then evaluate task and reward integrity, rollout–training faithfulness, and training observability, and §4.6 evaluates system efficiency.

4.1 Experimental Setup

Models and agent scaffolds. We use LEGO-RL to train **Qwen3.5-35B-A3B** with the VeOmni hybrid engine under three coding-agent harnesses: OpenHands SDK (Wang et al., 2025b), Claude Code (Anthropic, 2025), and OpenCode (SST, 2025). Training uses group-relative advantage estimation with the GSPO sequence-level policy loss (Zheng et al., 2025), implemented with verl and vLLM serving. The rollout temperature is 1.0 and the context budget is 200k tokens. Fully asynchronous runs use a maximum policy staleness of 1 and recover partial rollouts across weight synchronization. Termination-aware trajectory handling is applied before optimization as described in §3.2.3. Appendix I reports the complete resolved hyperparameters.

Datasets and evaluation. Training tasks are drawn from OpenSWE candidate pools. Unless otherwise specified, production runs use a 2,699-task OpenSWE-derived index produced by the task-selection pipeline evaluated in §4.3.¹ The training set is strictly disjoint from *SWE-bench Verified* at both the repository and

¹The training index is released at huggingface.co/datasets/LegoX/Lego-RL-2699.

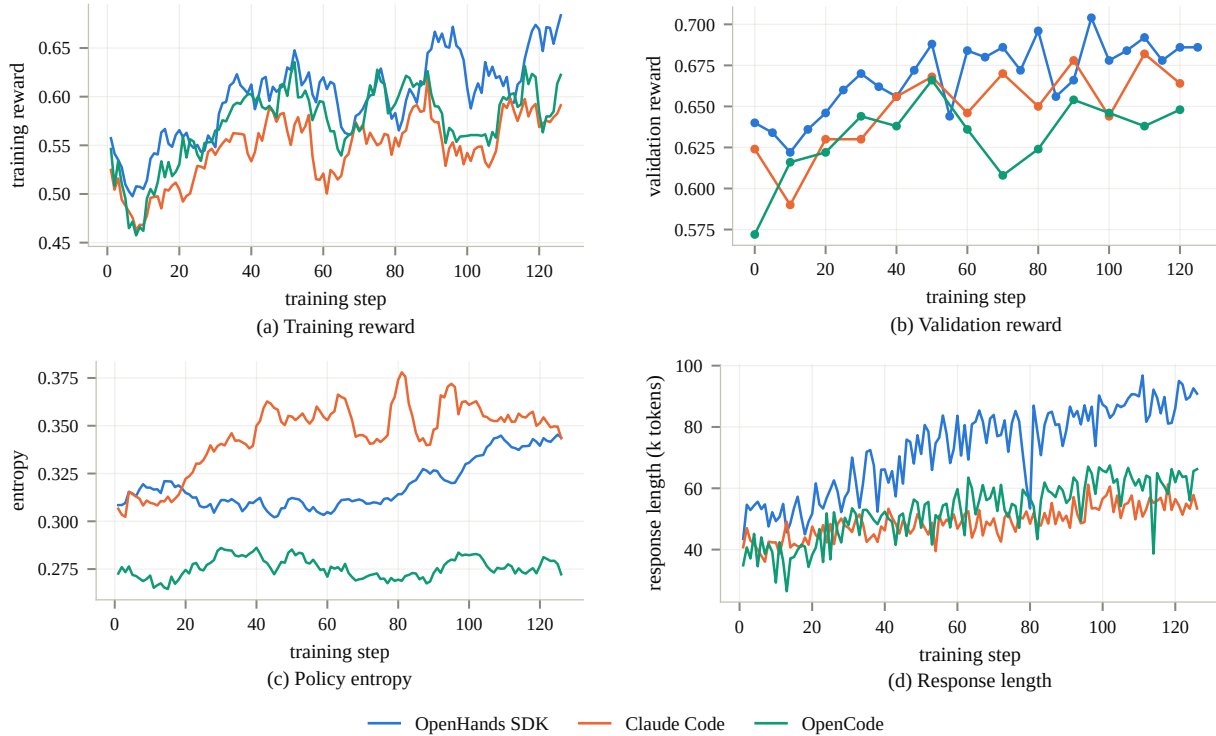


Figure 3: Training behavior of OpenHands SDK, Claude Code, and OpenCode over three epochs (126 training steps), showing training reward, validation reward, policy entropy, and mean response length.

instance levels, preventing overlap between training and evaluation tasks. Checkpoints are evaluated on the fixed *SWE-bench Verified* benchmark at temperature 0.7 using offline executable verification, and we report solve rate as the validation score.

4.2 Training Effectiveness

End-to-end effectiveness across agent scaffolds. Figure 3 compares training with OpenHands SDK, Claude Code, and OpenCode using the same Qwen3.5-35B-A3B initial checkpoint, 2,699-task training index, and 200k-token context budget. Training verifier reward increases under all three scaffolds, and each run improves on *SWE-bench Verified*. Validation scores peak at 0.704 with OpenHands SDK, 0.682 with Claude Code, and 0.666 with OpenCode, compared with step-0 scores of 0.640, 0.624, and 0.572, corresponding to gains of 6.4, 5.8, and 9.4 percentage points, respectively. Because the same initial policy yields different step-0 scores under different scaffolds, improvements are measured relative to each scaffold’s own baseline rather than compared directly across scaffolds. Policy entropy remains stable without collapse, while mean response length increases under all three scaffolds and most strongly under OpenHands SDK, showing that the scaffold substantially affects the trajectory distribution. We further analyze the behavioral change and system efficiency in §4.5 and §4.6.

Comparison with stronger baselines. Table 2 adds two reference points measured under the same protocol: Qwen3.6-35B-A3B, the next base generation, and KAT-Coder-V2.5-Dev (KwaiKAT Team, 2026), post-trained from it with supervised fine-tuning and RL. LEGO-RL-Qwen3.5-35B-A3B is the strongest model in all three harnesses, exceeding the newer base by 3.0, 4.8, and 6.0 points—more than the 3.4, 1.0, and 3.4 points the base generation itself is worth. The gains are also harness-specific: against its own base,

Coding agent	Model	SWE-bench Verified (%)
OpenHands SDK	Qwen3.5-35B-A3B (Qwen Team, 2026a)	64.0
	Qwen3.6-35B-A3B (Qwen Team, 2026b)	67.4
	KAT-Coder-V2.5-Dev (KwaiKAT Team, 2026)	67.0
	LEGO-RL-Qwen3.5-35B-A3B	70.4 (+6.4)
Claude Code	Qwen3.5-35B-A3B (Qwen Team, 2026a)	62.4
	Qwen3.6-35B-A3B (Qwen Team, 2026b)	63.4
	KAT-Coder-V2.5-Dev (KwaiKAT Team, 2026)	66.8
	LEGO-RL-Qwen3.5-35B-A3B	68.2 (+5.8)
OpenCode	Qwen3.5-35B-A3B (Qwen Team, 2026a)	57.2
	Qwen3.6-35B-A3B (Qwen Team, 2026b)	60.6
	KAT-Coder-V2.5-Dev (KwaiKAT Team, 2026)	64.8
	LEGO-RL-Qwen3.5-35B-A3B	66.6 (+9.4)

Table 2: SWE-bench Verified performance across the three coding agents. All numbers are measured by us under the same harness version and evaluation protocol (temperature 0.7, 200 turns, 200k context budget).

KAT-Coder-V2.5-Dev gains 3.4 points under Claude Code, the harness its authors report, but -0.4 under OpenHands SDK, where the untuned Qwen3.6-35B-A3B outscores it. We cannot isolate a cause, but the pattern is the one this work assumes: a gain obtained under one agent control flow need not survive another.

4.3 Task Reliability and Reward Integrity

Reliable RL depends on three properties: *task validity*, ensuring that tasks can be executed and graded correctly; *trajectory validity*, ensuring that rollout outcomes reflect policy behavior rather than infrastructure failures; and *reward informativeness*, ensuring sufficient within-group reward variation for effective policy optimization. LEGO-RL addresses these properties through task screening, termination-aware trajectory admission, and difficulty-aware task selection.

Task validity. LEGO-RL first applies static filtering to the 36,884 OpenSWE-derived candidates. Rule-based screening enforces basic validity, repository diversity, and coarse complexity constraints, reducing the pool to 22,806 tasks. Build and verifier validation further remove tasks that cannot be executed or graded reliably, leaving 21,681 tasks; approximately 2.5% of inspected tasks contain verifier logic that incorrectly applies the reference patch. The remaining tasks undergo rollout-based difficulty screening using Qwen3.6-27B with the OpenHands SDK scaffold. Retaining tasks solved 1–3 times in four trials produces the final 2,699-task training index. Although screening uses a single model–scaffold configuration, the resulting task pool also supports effective training with Claude Code and OpenCode.

Trajectory validity. Task-level validation cannot guarantee that every rollout produces a trustworthy training outcome. LEGO-RL therefore applies termination-aware admission before optimization: trajectories ending in infrastructure failures are excluded from group-relative advantage estimation and the policy loss, whereas valid trajectories reaching configured turn or token limits retain their verifier outcomes. Figure 4 shows that 7.1% of Claude Code, 2.4% of OpenHands SDK, and 6.4% of OpenCode trajectories are excluded from optimization. Termination profiles differ across scaffolds, with wall-clock timeouts dominating under Claude Code and environment-setup failures under OpenCode. These differences come from the harness as much as from the infrastructure: the same sandbox stack produces a different termination mix under each harness.

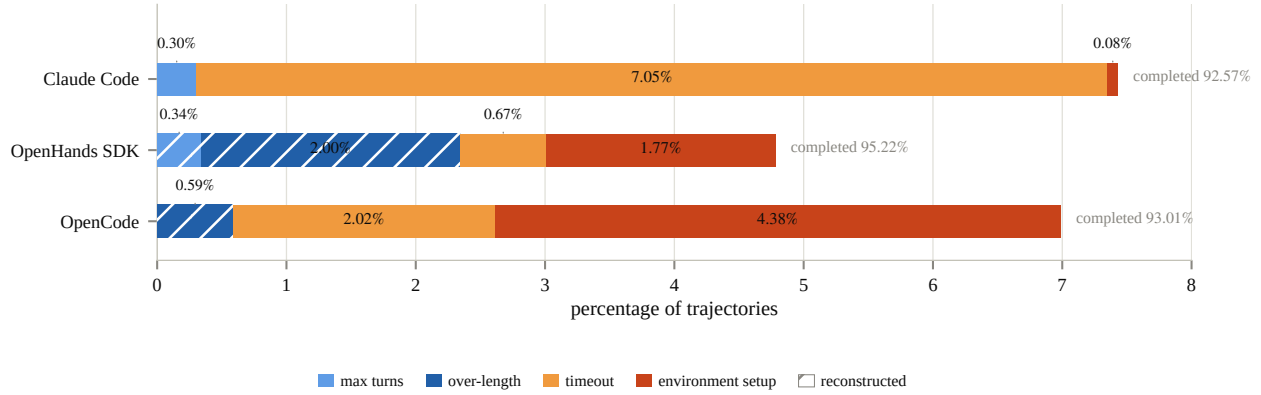


Figure 4: Trajectory termination profiles across agent scaffolds. Bars show the proportion of trajectories by termination reason; timeout and environment-setup failures are excluded from optimization.

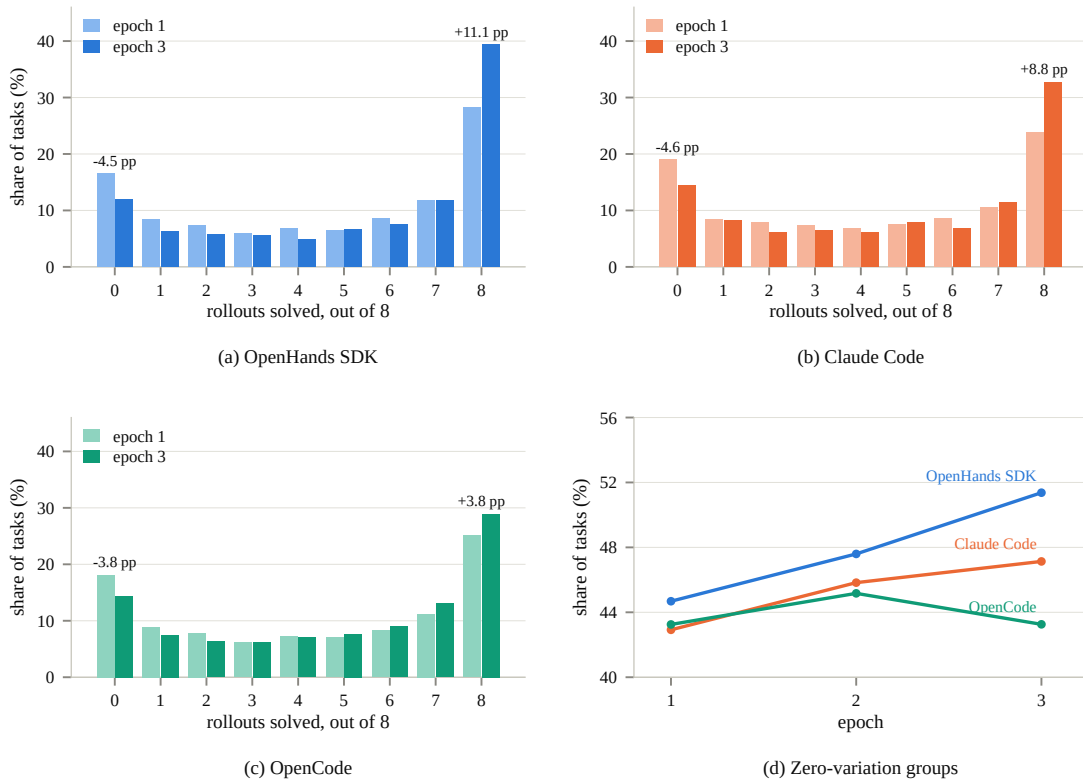


Figure 5: In-batch reward distributions across training. **(a–c)** Distribution of tasks by the number of successful rollouts out of eight in the first and last epoch for each scaffold. The 0/8 and 8/8 groups provide no group-relative advantage. **(d)** Combined proportion of these zero-variation groups across epochs.

Reward informativeness. With eight rollouts per task, groups solved either zero or eight times provide no within-group reward variation and therefore no group-relative learning signal. Comparing the first and last epoch (Figure 5), the proportion of all-wrong groups decreases while the proportion of all-solved groups increases across all scaffolds, showing that the informativeness of a fixed task pool changes as the policy

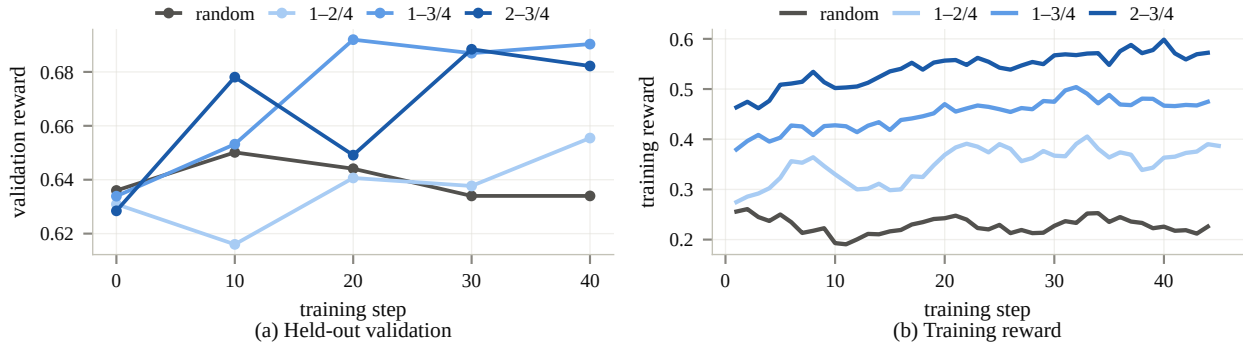


Figure 6: Task-selection ablation across four 951-task pools. **(a)** Held-out validation solve rate, measured over the validation tasks that executed. **(b)** Training verifier reward; levels are pool-specific, so only the slopes are comparable.

improves. Under OpenHands SDK, the proportion of zero-variation groups increases from 44.7% to 51.4%, as the growth in all-solved groups outweighs the decline in all-wrong groups. Under OpenCode, the smaller increase in all-solved groups leaves the zero-variation proportion approximately unchanged at 43.3%. These results show that task difficulty is policy-relative and that a fixed task pool can gradually provide less informative group-relative supervision.

Effect of difficulty screening. We further isolate the role of difficulty screening by comparing four 951-task training pools under otherwise matched configurations: the lower and upper halves of the selected difficulty band, the full band, and a random sample from the unscreened pool. A validation task whose rollout is lost to a harness failure is scored zero, not retried, and the four arms ran at different inference concurrency. We therefore report solve rates over the tasks that executed. This correction moves an arm by 0.6 to 5.2 percentage points, and it is what makes the arms comparable: all four initial passes evaluate the same checkpoint, and once corrected they agree to within 0.8 percentage points, against 2.4 before. Because a single pass still samples each task once, we compare post-warmup validation averages rather than individual passes. The full band and its upper half both improve, reaching post-warmup averages of 0.671 and 0.670, whereas the lower half reaches 0.640 and the unscreened pool improves on neither measure, ending at its starting level (Figure 6). During screening, 72.7% of the unscreened pool’s tasks are never solved and 13.4% are always solved, leaving only a small fraction capable of producing within-group reward variation. These results show that difficulty screening is important not merely for task quality, but for maintaining a sufficient density of tasks that provide usable group-relative learning signals.

4.4 Faithful Optimization

We next evaluate whether training preserves the policy behavior observed during rollout, focusing on token-level probability alignment, routing consistency for sparse models, and generation-time capture under harness-side history rewriting.

Rollout-training alignment. We compare π_{roll} , recorded by the proxy at the serving boundary, with π_{train} , recomputed by the trainer over the same assistant tokens. Across the tokens of a trajectory we report the Pearson correlation between the two log-probabilities, their mean absolute difference $|\Delta \log p|$, which is the log per-token importance ratio that enters the gradient, and the token-averaged $\text{KL}(\pi_{\text{roll}} \parallel \pi_{\text{train}})$ under the k_3 estimator (Schulman, 2020); all three are standard checks on train-inference mismatch (Fu et al., 2025). Alignment remains consistently high across OpenHands SDK, Claude Code, and OpenCode: the

Coding agent	Pearson r	KL ($\times 10^{-3}$)	$ \Delta \log p $ per trajectory ($\times 10^{-3}$)		
			p50	p90	p99
OpenHands SDK	0.9993	0.75	0.7	1.2	2.1
Claude Code	0.9980	1.35	0.7	1.3	2.7
OpenCode	0.9993	0.60	0.6	1.1	2.0

Table 3: Rollout-to-training alignment over the three matched production runs: probabilities captured at the serving boundary against trainer-side recomputation over the corresponding assistant tokens. All statistics are medians over training steps.

median Pearson correlation is at least 0.998 for every scaffold and never falls below 0.989 at any training step, while the p99 per-trajectory discrepancy in mean token log-probability remains below 3×10^{-3} (Table 3). These results show that trainer-side probability recomputation closely reproduces rollout-time generation across different agent scaffolds and context-management policies.

Routing consistency. For sparse policies, reproducing the same token sequence is insufficient when rollout and training select different experts. Replaying rollout-time routing decisions increases rollout-training correlation from 0.9946 to 0.9993 and reduces the mean token log-probability discrepancy from 0.0062 to 0.0025 on the same single-node workload. At the first training step, where serving and trainer weights are identical, expert overlap reaches 0.996 with 0.985 top-1 agreement. Deliberately misaligned replay degrades all alignment measures, confirming that routing decisions must remain associated with the tokens generated under them; additional negative-control results are reported in Appendix B.

Generation-time capture and history consistency. Harness-side history processing makes post-hoc trajectory reconstruction unreliable, motivating capture at generation time. In Claude Code production trajectories, tool-call reserialization is the most common apparent mismatch: matching tool calls by identifier rather than serialized arguments resolves 207 of 222 (93%) cases without altering the captured tokens. Sub-agent requests may also be mixed with the parent trajectory when they share a session; isolating them into separate sessions eliminates this issue in a subsequent 27-trial check, from 6.3% to 0%. Trajectories interrupted by weight synchronization are retained only when the captured prefix remains exact, whereas histories that are genuinely rewritten or truncated, such as through context compaction, are excluded from optimization. Training is therefore restricted to tokens whose rollout-time identity and probability information can be preserved exactly.

4.5 Observability and Behavioral Analysis

Trajectory-level observability enables both failure diagnosis and behavioral analysis by linking aggregate training metrics to task-, trajectory-, and execution-level evidence.

Failure diagnosis. The Live UI connects changes in training metrics to the underlying execution and trajectory evidence (Figure 7). In one run, validation reward fell from 0.556 to 0.150, while only 60 of 172 trajectories reached verification; termination analysis traced the failure to task setup rather than policy degradation. In another run, all 1,024 trajectories terminated after a single turn, and trajectory inspection identified an incompatible tool-call parser. In a separate collapsed run, trajectory-level analysis revealed the disappearance of valid tool calls and yielded an early-stop condition that would have triggered eight steps before termination. Together, these cases show that the observability layer can distinguish policy degradation from failures in execution, task setup, and agent integration.

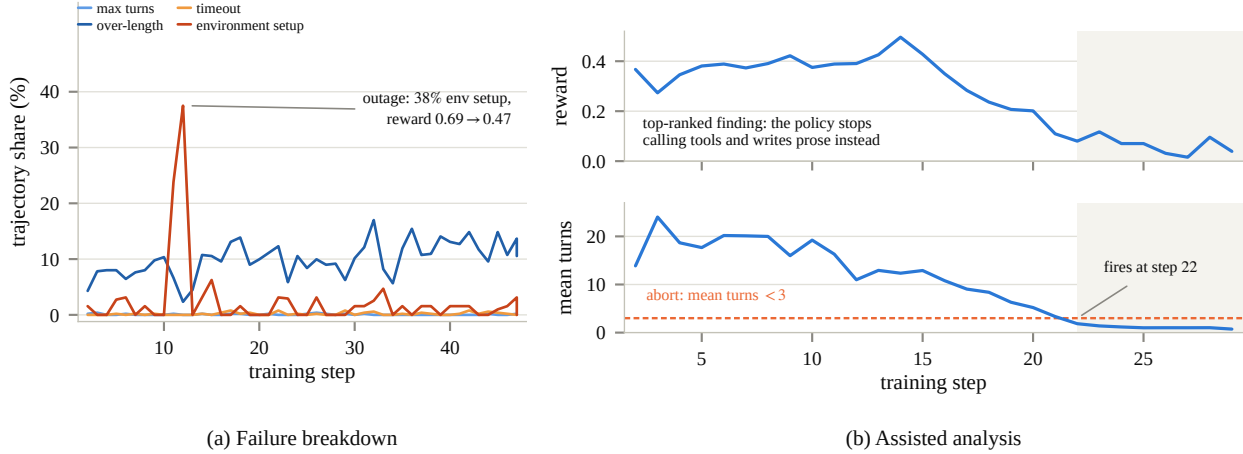


Figure 7: **Failure diagnosis with the Live UI.** (a) Per-step termination reasons for an environment-failure run. (b) Assisted analysis of a collapsed run and the corresponding early-stop condition. (a) and (b) are two different diagnostic runs, neither is the Claude Code production run reported elsewhere in this section.

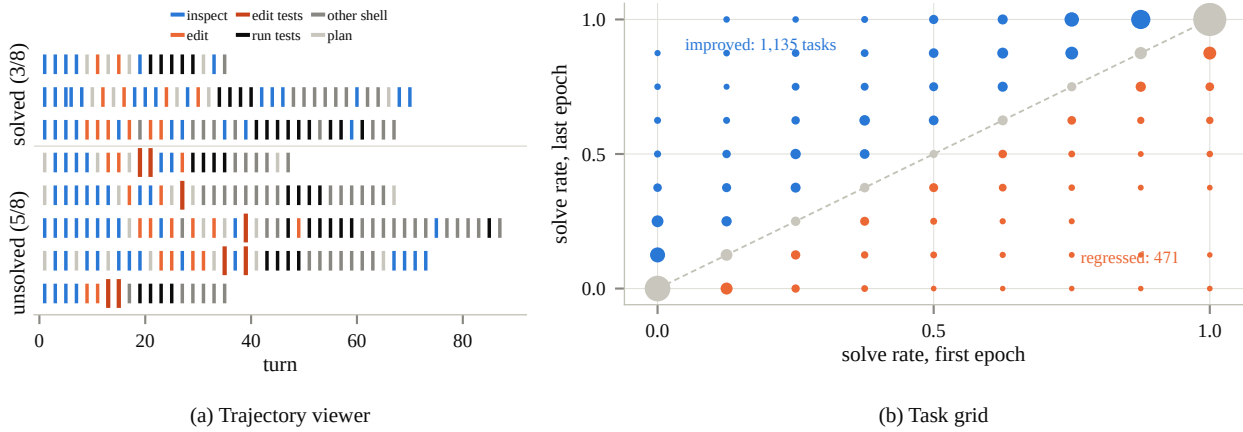


Figure 8: **Behavioral analysis with the Live UI, Claude Code run.** (a) Tool-use trajectories for eight rollouts of one task. (b) Task-level solve rates in the first and last sampled epochs.

Task-level heterogeneity. Mean reward improvement can mask substantial variation across individual tasks. The Live UI exposes this variation at task and trajectory granularity (Figure 8). Between the first and last sampled epochs, 1,135 tasks in the Claude Code run increase in observed solve rate while 471 decrease; OpenHands SDK shows a similar pattern, with 1,136 increasing and 445 decreasing. The median increase is 0.250, compared with a median decrease of 0.125. Because solve rates are estimated from eight asynchronously sampled rollouts per task, these differences are interpreted as observed task-level changes rather than direct measures of learning or forgetting.

Changes in self-checking and error recovery. In the OpenHands SDK run, agents increasingly inspect their work before and after editing. The proportion of trajectories that reread a file after modifying it rises from 73.6% to 98.1%, while the number of distinct files examined before the first edit increases from 3.5 to 6.9. By contrast, recovery after intermediate command failures changes only modestly: among affected trajectories, the proportion that ultimately solve the task increases from 63.9% to 66.8%. The larger shift in self-checking suggests that training changes verification behavior more strongly than recovery after an error

Stage	Mean (s)	p50	p90	p99	Mean-time fraction
Sandbox setup	21.6	7.7	41.8	275.2	2.3%
Agent setup	4.2	4.0	4.8	8.4	0.5%
Agent execution	840.5	708.4	1604.3	2801.1	91.3%
Verification	35.9	4.1	29.6	928.5	3.9%
Other	20.3	8.1	12.3	965.5	2.2%
Trial total	920.4	770.0	1742.7	3189.1	100%

Table 4: Stage-wise wall-clock statistics across 3,699 OpenHands SDK training trials. Mean-time fraction is computed relative to the mean total trial duration; Other includes scheduling, trajectory handling, and sandbox cleanup.

has already occurred. Appendix G reports the full trajectory-level analysis.

Growth in interaction horizon. Response-length growth is driven primarily by more interaction steps rather than longer individual turns. In the OpenHands SDK run, mean response length increases from 43.5k to 90.9k tokens, while turns per trajectory increase from 46.6 to 83.1 (+78%) and tokens per turn from approximately 933 to 1,093 (+17%). The increase is smaller under Claude Code, where mean response length grows from 41k to 51k under the same model, task pool, and context budget. The realized interaction horizon therefore depends on both the policy and the agent scaffold, with corresponding system-level consequences analyzed in §4.6. Additional analyses of tool allocation, response composition, and validation failures are reported in Appendices F, E, and D.

4.6 System Efficiency

We evaluate whether LEGO-RL can sustain long-horizon coding-agent RL at practical throughput by examining end-to-end runtime, rollout scheduling, sandbox setup, and image delivery.

End-to-end bottleneck and pipeline balance. Agent execution dominates the cost of long-horizon coding-agent RL. Across 3,699 OpenHands SDK trials, agent execution accounts for 91.3% of mean wall-clock time, with a mean duration of 840.5s out of 920.4s per trial (Table 4). Sandbox setup and verification account for only 2.3% and 3.9% on average, although both exhibit substantial tail latency. Under asynchronous training, the median fraction of idle rollout slots is zero for both OpenHands SDK and Claude Code. The trainer nevertheless waits for rollouts for 40.8% and 66.1% of training time, respectively, indicating that rollout generation rather than optimization is the primary throughput bottleneck. This imbalance becomes more pronounced as agent trajectories grow longer.

Asynchronous scheduling. Agentic rollouts vary substantially in duration, causing synchronous generation to be delayed by the slowest trajectories. LEGO-RL instead schedules rollouts asynchronously, allowing new trajectories to begin without waiting for the preceding batch to complete. In an offline synchronous screening workload using the same agent, sandbox, and verification stack, the slowest 10% of trajectories account for 24.5% of total trajectory work-time. Reconstructed progress timestamps reveal 31 batch-boundary stalls, with a median duration of 38.7 minutes and a maximum of 135.9 minutes. Asynchronous production training avoids these synchronization delays and keeps rollout capacity continuously utilized. The synchronous workload is an offline screening campaign, not a matched training run, so these numbers quantify synchronization overhead and not end-to-end training speedup.

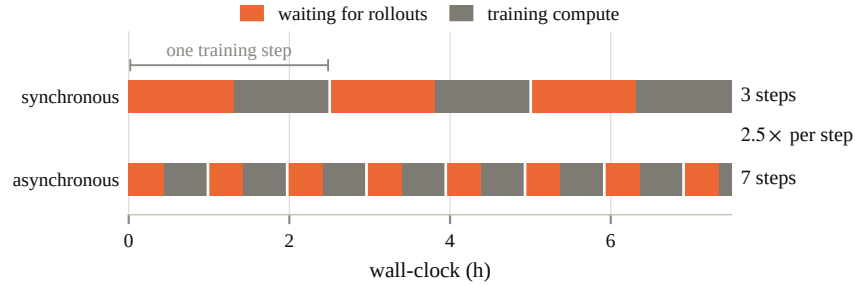


Figure 9: Trainer schedule under synchronous and asynchronous execution.

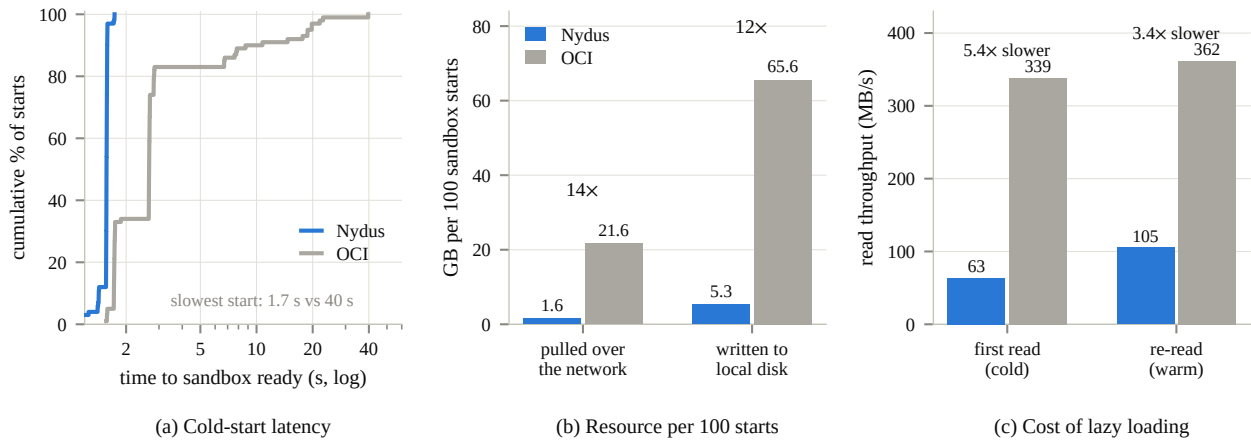


Figure 10: Lazy versus full image delivery over the same 100 task images. Nydus streams image chunks on demand, whereas OCI denotes the conventional pull, which materializes the entire image before the container starts. The panels report startup latency, cumulative network and disk traffic, and in-container read throughput.

Synchronous-asynchronous comparison. We directly compare synchronous and asynchronous training under matched configurations. Over the same 7.5 hours, synchronous training completes three steps, whereas asynchronous training completes seven, corresponding to a measured 2.5 $\times$ reduction in step time (Figure 9). The two runs, however, use GPU groups with different optimizer throughput: per trained token, the synchronous run spends 2.1 $\times$ longer on optimization. After correcting for this compute-rate difference, the estimated synchronous step time decreases from 2.5 to 1.9 hours, compared with 1.0 hour under asynchronous execution. This result is specific to the staleness-1 setting used here, while larger staleness may enable greater rollout-optimization overlap.

Sandbox setup. Reducing repeated environment construction substantially lowers sandbox startup overhead. Prebuilt task images achieve a median paired speedup of 33.2 $\times$ over inline Dockerfile execution, while mounting the coding-agent runtime instead of installing it inside each sandbox yields a 15.4 $\times$ speedup. Table 5 summarizes these ablations together with the image-delivery optimization described below. Packaging grading dependencies, by contrast, provides no measurable latency benefit and is retained primarily to ensure reward reproducibility.

Optimization	Stage	Median latency		Paired ratio		n
		With	Without	Median	p10–p90	
Lazy image pull	sandbox setup	1.57 s	2.66 s	1.7×	—	100
Prebuilt task image	sandbox setup	1.04 s	36.2 s	33.2×	17.9–67.5×	50
Mounted agent runtime	agent setup	0.51 s	7.82 s	15.4×	14.5–16.6×	50
Packaged grading toolchain	verification	3.81 s	2.72 s	0.71×	0.67–0.73×	50

Table 5: Ablation of sandbox optimizations.

Lazy image delivery. Lazy image delivery reduces the remaining startup cost by fetching image data on demand rather than pulling the full task image before execution. Across 100 SWE-bench Verified task images, lazy pull improves median cold-start latency by 1.7× and maximum latency by 23×, while reducing cumulative network traffic from 21.6 GB to 1.59 GB and disk writes from 65.6 GB to 5.29 GB (Figure 10). The gains are largest when coding-agent workloads access only a small fraction of the image and disappear on warm starts, where both delivery paths perform similarly. The trade-off is lower uncached in-container read throughput because missing blocks are fetched remotely. In production, we additionally use a shared snapshotter daemon to prevent the daemon accumulation observed with the default per-image mode, trading per-image isolation for lower long-term resource overhead.

5 Conclusion

We present LEGO-RL, a framework that connects native coding-agent harnesses to scalable policy-gradient optimization while preserving their original control flows. By integrating sandboxed execution and verification, token-faithful rollout capture, asynchronous training, reward-integrity safeguards, and trajectory-level observability, LEGO-RL supports reliable and faithful training across OpenHands SDK, Claude Code, and OpenCode. Together, these results highlight that scaling coding-agent RL requires not only scalable optimization, but also reliable execution environments, faithful trajectory capture, and feedback mechanisms that evolve with increasingly capable agents.

Limitations and Future Work

Limitations. Our evaluation has several limitations. First, all experiments use Qwen3.5-35B-A3B, and each coding-agent harness is trained separately, so generalization to other model architectures and mixed-harness training remains to be evaluated. Second, the cost of production-scale training limits each main configuration to a single run, leaving run-to-run variance in training gains and system-level efficiency unquantified. Third, executable verification provides a reliable but coarse binary reward and cannot assign intermediate credit to behaviors such as error recovery. Our reward-integrity defenses address the failure modes observed in our experiments but do not guarantee robustness to all possible reward-exploiting strategies. Finally, the reported sandbox and image-delivery speedups depend on the deployment environment and should be interpreted as measurements of our implementation rather than universal properties of LEGO-RL. More broadly, the diagnostic analyses presented here identify plausible causes from execution and trajectory evidence, but do not constitute automated causal verification.

Future work. Several of these limitations point at work already under way. We are extending LEGO-RL to mixed training: one policy over a task pool that mixes repository repair with other verifiable software tasks, and one policy trained across several harnesses at once rather than one run per harness—the latter

motivated by the harness dependence visible in Table 2. We are also adding adapters for further harnesses, richer credit assignment than terminal binary rewards, and automated diagnosis in the Live UI. LEGO-RL is developed in the open; we will continue to release framework updates, harness adapters, trained checkpoints, and task indices.

References

- Agentica Team and Together AI. DeepSWE: Training a fully open-sourced, state-of-the-art coding agent by scaling RL. <https://www.together.ai/blog/deepswe>, 2025.
- Anthropic. Claude code. <https://github.com/anthropics/claude-code>, 2025.
- Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *arXiv preprint arXiv:2505.20411*, 2025.
- Berkeley Sky Computing Lab. rLLM: Democratizing reinforcement learning for LLMs. <https://github.com/rllm-org/rllm>, 2026.
- Shiyi Cao, Dacheng Li, Fangzhou Zhao, Shuo Yuan, Sumanth R. Hegde, Connor Chen, Charlie Ruan, Tyler Griggs, Shu Liu, Eric Tang, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. SkyRL-Agent: Efficient rl training for multi-turn llm agent. *arXiv preprint arXiv:2511.16108*, 2025.
- Mouxian Chen, Lei Zhang, Yunlong Feng, Xuwu Wang, Wenting Zhao, Ruisheng Cao, Jiaxi Yang, Jiawei Chen, et al. SWE-Universe: Scale real-world verifiable environments to millions. *arXiv preprint arXiv:2602.02361*, 2026.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, et al. SWE-Bench Pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- Dayuan Fu, Shenyu Wu, Yunze Wu, Zerui Peng, Yaxing Huang, Jie Sun, Ji Zeng, Mohan Jiang, Lin Zhang, Yukun Li, Jiarui Hu, Liming Liu, Jinlong Hou, and Pengfei Liu. daVinci-Env: Open SWE environment synthesis at scale. *arXiv preprint arXiv:2603.13023*, 2026. Presents the OpenSWE environment collection, the name used in this report.
- Wei Fu et al. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.
- Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Sergey Abramov, Andrei Andriushchenko, Filipp Fisin, Sergei Skvortsov, and Boris Yangel. Training long-context, multi-turn software engineering agents with reinforcement learning. *arXiv preprint arXiv:2508.03501*, 2025.
- Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents. *arXiv preprint arXiv:2504.07164*, 2025.

- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- KwaiKAT Team. KAT-Coder-V2.5 Technical Report, July 2026. URL <https://arxiv.org/pdf/2607.05471>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023. vLLM.
- Xufang Luo et al. Agent Lightning: Train any ai agents with reinforcement learning. *arXiv preprint arXiv:2508.03680*, 2025.
- Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing MoE reinforcement learning by aligning training and inference routers. *arXiv preprint arXiv:2510.11370*, 2025.
- NVIDIA NeMo. MOLT: A PyTorch-native reinforcement learning framework for agentic research. <https://github.com/NVIDIA-NeMo/labs-molt>, 2026.
- Jiayi Pan, Xingyao Wang, et al. Training software engineering agents and verifiers with SWE-Gym. *arXiv preprint arXiv:2412.21139*, 2024.
- Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026a. URL <https://qwen.ai/blog?id=qwen3.5>.
- Qwen Team. Qwen3.6-35B-A3B: Agentic coding power, now open to all, April 2026b. URL <https://qwen.ai/blog?id=qwen3.6-35b-a3b>.
- John Schulman. Approximating KL divergence, 2020. URL <http://joschu.net/blog/kl-approx.html>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. Introduces Group Relative Policy Optimization (GRPO).
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient rlhf framework. In *European Conference on Computer Systems (EuroSys)*, 2025. verl.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- SST. OpenCode. <https://github.com/sst/opencode>, 2025.
- Lintang Sutawika, Aditya Bharat Soni, Bharath Sriraam R R, Apurva Gandhi, Taha Yassine, Sanidhya Vijayvargiya, Yuchen Li, Xuhui Zhou, Yilin Zhang, Leander Melroy Maben, and Graham Neubig. Codescout: An effective recipe for reinforcement learning of code search agents. *arXiv preprint arXiv:2603.17829*, 2026.

- The Harbor Team. Harbor: A framework for evaluating and optimizing agents in containerized environments. <https://github.com/harbor-framework/harbor>, 2026.
- THUDM. slime: An LLM post-training framework for RL scaling. <https://github.com/THUDM/slime>, 2025.
- Weixun Wang, XiaoXiao Xu, Wanhe An, Fangwen Dai, Wei Gao, Yancheng He, Ju Huang, Qiang Ji, et al. Let it flow: Agentic crafting on rock and roll, building the ROME model within an open agentic learning ecosystem. *arXiv preprint arXiv:2512.24873*, 2025a. The ALE stack: ROLL trainer, ROCK sandbox manager, iFlow CLI agent.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025b. *arXiv preprint arXiv:2407.16741*.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025.
- Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Honglin Guo, Jiaqi Liu, Rui Zheng, Junjie Ye, et al. AgentGym-RL: Training LLM agents for long-horizon decision making through multi-turn reinforcement learning. *arXiv preprint arXiv:2509.08755*, 2025.
- Binfeng Xu, Hao Zhang, Shaokun Zhang, Songyang Han, Mingjie Liu, Jian Hu, Shizhe Diao, Zhenghui Jin, Yunheng Zou, Michael Demoret, Jan Kautz, and Yi Dong. Polar: Agentic rl on any harness at scale. *arXiv preprint arXiv:2605.24220*, 2026.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025.
- Xiao Yu, Baolin Peng, Ruize Xu, Hao Zou, Qianhui Wu, Hao Cheng, Wenlin Yao, Nikhil Singh, Zhou Yu, and Jianfeng Gao. OpenForgeRL: Train harness-native agents in any environment. *arXiv preprint arXiv:2607.21557*, 2026.
- Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. Multi-SWE-bench: A multilingual benchmark for issue resolving. *arXiv preprint arXiv:2504.02605*, 2025.
- Guibin Zhang, Hejia Geng, Xiaohang Yu, Zhenfei Yin, Zaibin Zhang, et al. The landscape of agentic reinforcement learning for LLMs: A survey. *arXiv preprint arXiv:2509.02547*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

A Reward-Integrity Failure Modes

The reliability of our training pipeline depends on the verifier reward faithfully reflecting task completion. During development, we identified six systematic failure modes that violate this assumption. Table 6 categorizes these failures by their root cause—whether the agent exploits the reward mechanism, or the environment produces a reward decoupled from agent behavior—and lists the corresponding defenses deployed in production.

Failure Mode	Incidence	Defense
<i>Agent-side: shortcut exploitation</i>		
Reads git history	4.6–20.5%	Rebase history to a single commit during agent phase; restore before grading
Downloads reference fix	1.9%	Per-phase egress firewall in privilege-separated side-car
Edits test files	2.4–19.4%	Withhold tests until grading; revert test-path edits
<i>Environment-side: reward detached from agent</i>		
Grader applies reference patch	2.5%	Audit affected instances out of pool; flag live if degenerate reward propagates
Grader requires network access	—	Package all grade-time dependencies; ensure deterministic invocation
Incomplete repository build	—	Hermetic fail-fast build; report setup failure explicitly, not as zero reward

Table 6: Reward-integrity failure modes and their mitigations. Incidence rates are measured prior to deploying the defenses; “—” indicates cases not separately quantified.

B Routing-Replay Negative Control and Capture Coverage

The routing-replay mechanism introduced in §4.4 is subject to two independent failure modes: incorrect alignment whether a replayed routing decision is assigned to the correct token. The second concerns completeness: whether a routing decision was recorded at all. These are independent failures—a system can have perfect alignment but incomplete capture, or complete capture but systematic misalignment.

Misaligned replay. An intermediate implementation introduced a systematic one-position offset between each token and its replayed routing decision, routing every token through the experts selected for its neighbor. Table 7 shows that this misalignment degrades all metrics relative to disabling replay entirely. Crucially, this degradation is not detectable by merely verifying that the replay mechanism is active—the system appears to function normally. The only diagnostic that separates correct from incorrect replay is a direct comparison of the replayed expert assignments against the model’s own online selections, which is why we report expert overlap and top-1 agreement in the Table 7.

Incomplete capture. For replay to be complete, routing decisions must be recorded for every generated token. Our initial capture buffer, however, was sized using a formula that underestimated the requirement for hybrid-attention models by roughly a factor of four. Worse, the out-of-range guard recorded excess decisions as zero rather than raising a failure, making the problem silent. Coverage therefore decayed with sequence length, reaching only 24% overall before diagnosis. After resizing the buffer, both production runs now exceed 99.8% coverage. The residual misses occur only on the longest sequences and degrade

Routing replay configuration	Pearson r	Mean $ \Delta p $	Expert overlap	Top-1 agreement
Disabled	0.9946	0.0062	—	—
Enabled, misaligned	0.7503	0.0954	0.083	0.026
Enabled, aligned	0.9993	0.0025	0.996	0.985

Table 7: Routing-replay configurations compared. Expert overlap and top-1 agreement are undefined when replay is disabled.

gracefully: tokens without a recorded decision are replayed unconstrained, rather than being forced through an incorrect assignment. This fail-soft behavior ensures that incomplete capture, unlike misalignment, does not actively harm the policy.

C Assisted-Analysis Case Study: A Collapsed Run

The run summarized in Figure 7(b) serves as an end-to-end illustration of the assisted-analysis pane. The configuration is Qwen3-30B-A3B with the OpenHands SDK on a 449-task pool; training reward fell from 0.351 to 0.050 and held-out validation reward from 0.230 to 0.014 over 29 logged steps. The text that follows is the pane’s output for this run, edited only for formatting; each claim was verified against the raw trajectories before inclusion. The pane is required to mark missing data sections as such rather than infer them; for this run, three sections—zero-advantage accounting, first-versus-last trajectory shape, and per-task solve statistics—were not populated. Table 8 reports the numeric evidence supplied to the pane: the mean of the first five logged steps against the mean of the last five, with a least-squares trend statistic (t) computed over the entire run.

Series	First five steps	Last five steps	t
training reward	0.351	0.050	−7.5
turns per trajectory	18.9	0.96	−14.8
response length (tokens)	12,170	2,276	−7.7
tokens per turn	650	2,397	—
policy entropy	0.134	0.230	+7.2
KL term of the loss	0.0046	0.130	+3.4
rollout–training agreement	0.995	0.970	−3.3
gradient norm	0.31	0.034	+0.4

Table 8: Evidence supplied for the collapsed run. t is the least-squares slope divided by its standard error over all logged steps; the gradient norm does not clear $|t| = 2$ and is therefore read as noise. Tokens per turn is a ratio of two rows above it and carries no separate trend statistic.

The pane identifies step 17 as the point where learning ceased. The training reward decline ($t = -7.5$) is mirrored by validation reward, ruling out a training-metric artifact. The proximate failure is the policy’s cessation of action: between steps 17 and 25, turns per trajectory fell from 9.0 to 1.0 and did not recover, response length shrank to one-fifth of its prior value, and reward dropped to a tenth. Trajectory inspection confirms the mechanism: a representative late trajectory contains a single turn with no tool call, with intended shell commands appearing as fenced code blocks in the model’s prose. Rising entropy ($t = +7.2$) over the same interval is consistent with degeneration, not exploration.

The group-level advantage structure collapses as nearly every group becomes all-wrong: advantage bounds reach zero at two of the last three steps, while the KL term rises to $150\times$ its initial level, leaving the update dominated by penalty rather than reward. Rollout–training agreement declines from 0.995 to 0.851

at step 22. The evidence does not distinguish whether this capture regression contributed to the collapse or merely reflects a policy far from its reference.

Infrastructure failures do not explain the collapse: setup-failure share never exceeded 1.6% of rollouts across steps 17–28, and thus postdates the collapse by more than ten steps. The case yields several monitoring criteria: a mean turn count approaching one is terminal; a batch with no partially solved group reached 100% at two of the last three steps; and rollout–training agreement crossed below 0.95 at step 21, eight steps before manual termination.

D How Validation Attempts Fail

A verifier reward records only whether a validation attempt succeeded. To see what changes among the attempts that do not, we classify every trajectory of a validation event by how far the attempt got: whether it edited the right file, edited it without fixing the issue, produced no edit, or exhausted its budget. Over the OpenHands SDK production run, comparing the first and last of its 26 validation events on the same 500-task set, the resolved share rises from 63.8% to 68.6% and the remainder redistributes. Attempts that edit the wrong file halve, from 6.4% to 3.0% of the set, and attempts that edit the file the reference patch edits without resolving the issue fall from 26.2% to 21.8%. One category moves the other way, from 1.6% to 5.4%: trajectories that exhaust their budget in a repeated pattern without progress. Attempts that produce no edit at all fall from 1.6% to 0.8%, and environment or verifier failures hold at 0.4%. Over training, localization failures decrease, patch-quality failures remain dominant, and budget-exhaustion failures become more frequent—the same shift §4.5 reports on the training set, here visible in the validation set. The classification is rule-based rather than a model judgment, and its categories are not equally reliable: the localization and no-edit categories are read from the actions taken, whereas budget exhaustion is inferred from repetition.

E Reasoning–Action Composition Across Training

We define the reasoning share of a response as the proportion of characters not contained within tool-call delimiters. Tool-call arguments are classified as acting rather than reasoning, since they correspond to the agent’s executable modifications to the codebase. (Scoring against visible prose alone would roughly double every level reported here.) The metric is computed on a fixed 120-task cohort subsampled from the SWE-bench Verified validation set and reused at every validation event of both runs, ensuring comparability across steps. Figure 11(a) reports both the per-task mean and the character-weighted mean; the two agree closely throughout, indicating that long trajectories neither dominate the average nor are discounted by it. The Claude Code curve in Figure 11(a) begins at a step-0 point from an earlier run of the identical configuration, as the production run’s own step-0 transcripts were not retained; the validation reward at that step matches the production run’s step-0 value.

Figure 11(a) shows that reasoning share rises over training under both scaffolds, from 0.22 to 0.39 (OpenHands SDK) and from 0.18 to 0.39 (Claude Code). The two scaffolds start apart but converge to the same level, pointing to a policy-level trend rather than a scaffold effect. This interpretation is observational: a shared training objective, task pool, and base model could equally produce the same pattern, and the measurement does not isolate a cause.

Panel (b) examines whether the increase is concentrated on tasks the run finds difficult. Grouping tasks by how many of their eight rollouts the run solved, the early-training reasoning share decreases with task difficulty; by the last epoch, all three groups sit at nearly the same level. Hard tasks gain the most simply because they start the lowest, not because reasoning grows without bound where it is needed. Since the grouping is based on post-training outcomes, this does not establish that additional reasoning causes

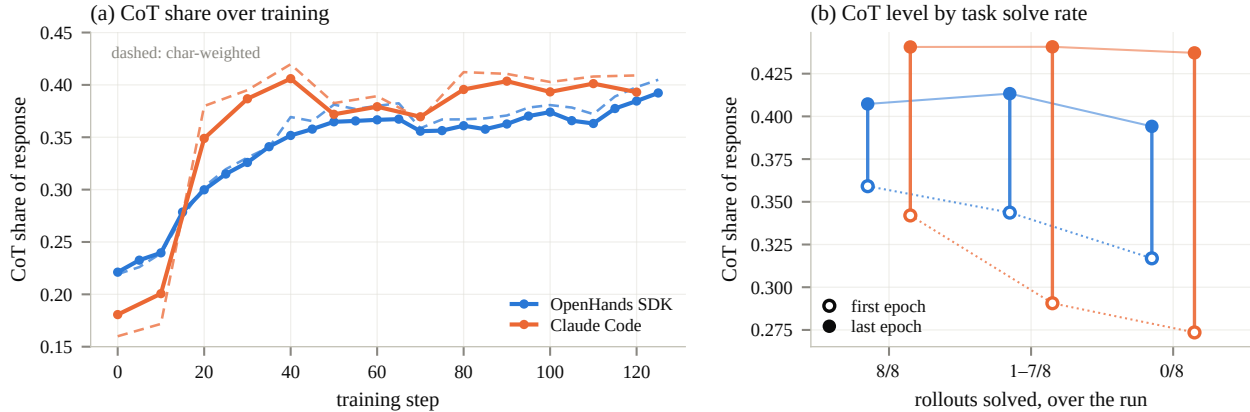


Figure 11: Reasoning share of the response over training on a fixed 120-task validation cohort. (a) Mean over tasks (solid) and character-weighted mean (dashed); (b) Median share at each task’s first (open) and last (filled) sampled epoch, grouped by number of rollouts solved.

success—that would require direct intervention on reasoning length. We therefore interpret the result as a drift in trajectory shape toward a common reasoning ratio, not as evidence of improved reasoning capability.

F Tool Allocation

The interaction horizon grows under both scaffolds: tool-calling turns per validation task rise from 69.9 to 106.7 (OpenHands SDK) and from 63.6 to 76.9 (Claude Code), as shown in Figure 12(a). Tool allocation shifts in scaffold-specific directions, but two trends are shared: test-suite invocation becomes more frequent and malformed calls decline. On a common file-operation axis, the initial difference between the two scaffolds narrows from 13.9 percentage points (23.9% vs. 37.8%) to 2.4 points (34.4% vs. 32.0%).

Panel (b) of Figure 12 provides the per-category breakdown over the first and last 420 trajectories of each run. OpenHands SDK reduces shell-based file inspection from 20.1% to 9.7% of calls while its structured view command rises from 14.5% to 24.6%; Claude Code moves in the opposite direction, with structured file tools falling from 37.8% to 32.0% and shell inspection rising from 6.5% to 7.3%. This comparison is approximate, however, as the two harnesses expose different tool sets; only Claude Code offers a typed search, so OpenHands SDK performs the equivalent work via shell commands. We therefore do not claim convergence of tool behavior in general.

Figure 12 thus captures the aggregate tool-use dynamics. The finer-grained agent behaviors that underlie these aggregates are presented in Table 9 below.

G Agent Behavior Changes Across Training

Table 9 is the full set of trajectory-level behaviors behind §4.5, each read off the agent transcript by a deterministic scan rather than by a judge. The upper block is self-verification and exploration, which improve substantially; the middle block is failure handling, which barely moves; the lower blocks give protocol compliance and the coverage/reliability split discussed in the main text.

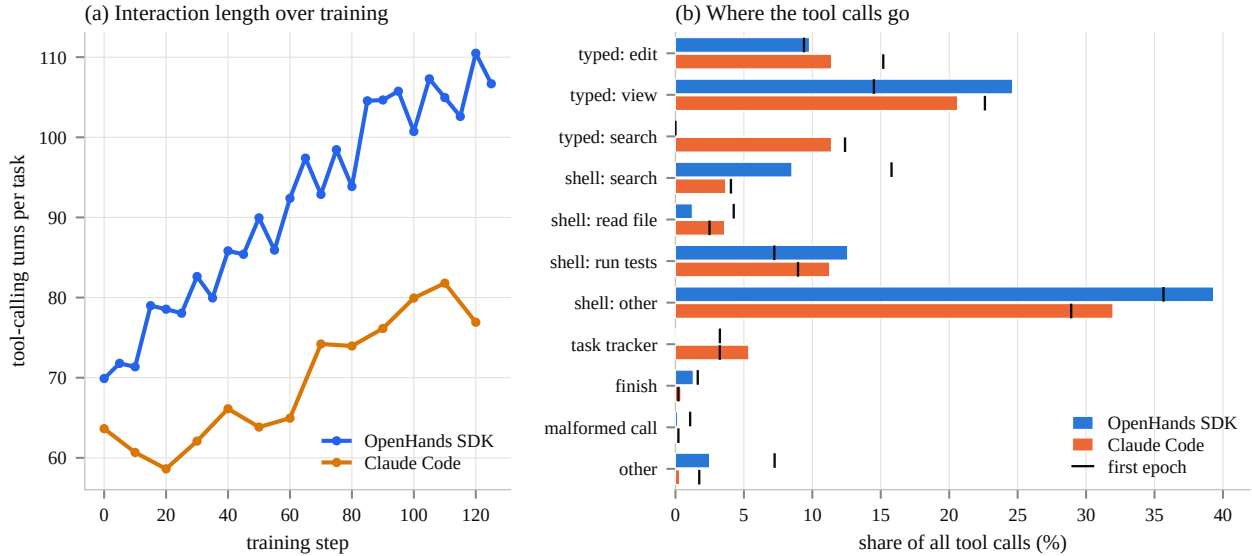


Figure 12: Agent behaviors before and after training, computed over 420 trajectories at each end of the production OpenHands SDK run. The $\text{pass}@k$ and pass^k rows are computed over prompt groups in the first and last third of the run.

Behavior	First	Last	Δ
Reads back a file it edited	73.6%	98.1%	+24.5
Runs the test suite	85.0%	93.6%	+8.6
Files explored before 1st edit	3.45	6.92	+3.47
Ends with an explicit finish	88.3%	91.9%	+3.6
Reproduces failure before editing	6.7%	11.2%	+4.5
Solves despite a failed command	63.9%	66.8%	+2.9
Malformed tool calls (of all calls)	1.07%	0.15%	-0.92
Coverage, $\text{pass}@8$	83.2%	87.9%	+4.7
Reliability, pass^8	28.3%	39.4%	+11.1

Table 9: Agent behaviors before and after training, over 420 trajectories at each end of the production OpenHands SDK run; the $\text{pass}@k$ rows are over prompt groups in the first and last third of the run.

H Live Observability Dashboard

Figures 13 and 14 show the observability UI of §3.3.2 on the Claude Code run analyzed in Figure 11, read while that run was still in flight (Qwen3.5-35B-A3B, 200k-token context budget, step 103); its shares therefore differ from the whole-run figures of §4.3.

In the per-instance task grid, each cell is one training task colored by its solve rate across the run; the header decomposes the aggregate reward into improved versus regressed tasks (first epoch vs. last), and the trajectory-shape strip tracks turn counts, token budgets, and the chain-of-thought share of responses as training proceeds. The termination-reason breakdown shows that 94.1% of 34,816 rollouts complete and carry learning signal, while timeouts and environment-setup failures are classified as environment noise and neutralized out of the loss.

Panel inventory. Beyond standard RL curves (entropy, KL, gradient norm, throughput, model-flops utilization), the dashboard provides:

- **Trajectory viewer.** Every trial’s full agent transcript rendered next to its verifier reward and termination reason, so a suspicious curve can be traced to concrete behavior.
- **Per-instance task grid.** Which task instances flip from unsolved to solved (or regress) across checkpoints, as an instance $\times$ checkpoint grid with a trend significance test.
- **Consistency panel.** The per-step fidelity suite of §4.4: probability Pearson, per-sample mean/max absolute log-ratio, the batch log-ratio histogram, and the ESS fraction under importance weighting.
- **In-batch distribution.** The share of the batch whose groups are partially solved and thus drive a group-relative update, plus the overlong ratio and length distribution.
- **Failure breakdown.** The per-step termination-reason mix together with per-stage timing and per-tool frequency—the first panels consulted when reward drops, since they separate infrastructure incidents from learning dynamics.
- **Validation failure inventory.** Each trajectory of a validation event classified by how far the attempt got, which turns a validation reward into a direction (Appendix D).
- **AI-assisted analysis.** An optional pane that summarizes a run’s metrics and trajectories with a language model acting as an RL diagnostician, surfacing candidate explanations for a reviewer to confirm (Appendix C works one case end to end).

Three supporting mechanisms make these views trustworthy during a live run. A central per-step *progress aggregator* collects a one-line summary from every finished trial (task, termination reason, reward, turns, wall-clock) and renders a live count with per-outcome tallies (“234/512 trials, 12 setup failures”), so an operator sees a step forming rather than waiting for its aggregate. When validation is suspected of scoring *false zeros* (a verifier artifact rather than a policy failure), an *offline regrade* path re-scores the affected trials outside the training loop and reconciles the curve, and infrastructure-failed trials can be purged from an index before they contaminate later analysis. Finally, any run view can be exported as a *static snapshot* and published as a standalone page, so a training incident can be shared and inspected without access to the cluster.

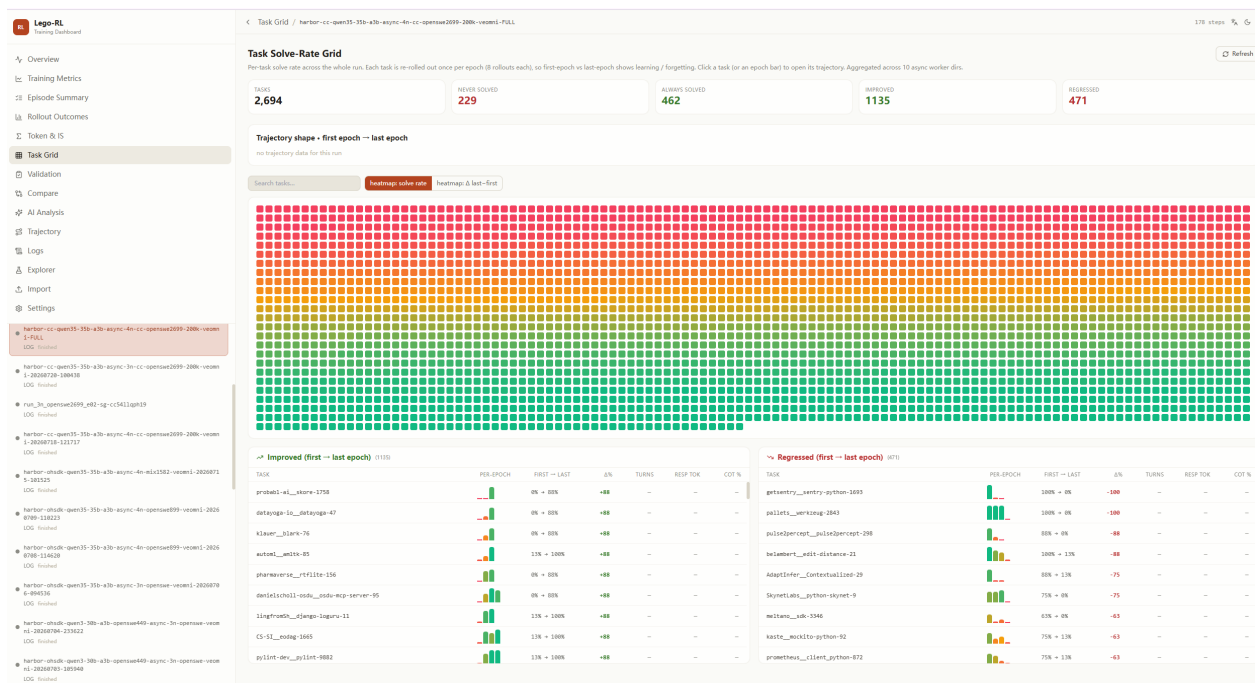


Figure 13: The per-instance task grid.



Figure 14: Diagnostic panels from the same run. *Top*: termination-reason breakdown per step. *Bottom*: the train-inference consistency panel.

I Run Configuration

Table 10 lists the hyperparameters shared by the three production runs reported in §4.2. Four entries require clarification. First, the KL term enters the loss only (never the reward) and uses the low-variance estimator. Second, the context budget is a split limit: prompt and response are truncated independently when either exceeds its respective share. Third, importance-sampling correction is disabled in all three runs because measured fidelity remained high throughout (§4.4); it is enabled only when log-probability diagnostics indicate drift. Fourth, the staleness threshold of 1 permits full asynchrony while bounding the maximum policy lag of any consumed rollout.

Policy loss	GSPO
Clip range (sequence-level)	$(3 \times 10^{-4}, 4 \times 10^{-4})$
Advantage estimator	GRPO
Loss aggregation	seq-mean-token-mean
KL reward penalty	none
KL loss coefficient	10^{-3}
Learning rate	1×10^{-6}
Learning-rate schedule	constant
Gradient clip	1.0
Prompts per batch	64
Rollouts per prompt	8
Micro-batch per GPU	1
Rollout temperature	1.0
Rollout top- p	1.0
Validation temperature	0.7
Validation samples per instance	1
Prompt budget	30k tokens
Response budget	170k tokens
Staleness threshold	1
Partial-rollout recovery	on
Importance-sampling correction	off
Training pool	2,699 tasks
Epochs	3

Table 10: Resolved hyperparameters of the three production runs (Qwen3.5-35B-A3B through the OpenHands SDK, Claude Code, and OpenCode).

J Data Formats

This appendix documents the three data representations a task passes through, using the real instance `12rambau__sepal_ui-814` from the OpenSWE-derived pool (long fields truncated).

Raw instance. Each upstream instance is a SWE-bench-style record: a repository snapshot, the issue text, the gold patch, the test patch, and the test lists that define the verifier outcome.

```
{
  "repo": "12rambau/sepal_ui",
  "instance_id": "12rambau__sepal_ui-814",
  "base_commit": "6d825ae167f96ad2e7b76b96ca07de562f74dcf0",
  "patch": "diff --git a/sepal_ui/sepalwidgets/alert.py ... (truncated)",
  "test_patch": "diff --git a/tests/test_sepalwidgets/test_Alert.py ... (truncated)",
  "problem_statement": "avoid to force developer to set total each time\n
  I should be able to init the progress of an Alert first and then
  simply update the progress. ... (truncated)",
  "FAIL_TO_PASS": ["tests/test_sepalwidgets/test_Alert.py::test_update_progress"],
  "PASS_TO_PASS": ["tests/test_sepalwidgets/test_Alert.py::test_init", ...],
  "environment_setup_commit": "b91b2a2c45b4fa80a7a0c699df978ebc46682260",
  "docker_image": "sweb.eval.x86_64.12rambau_1776_sepal_ui-814:latest",
  "install_config": {"install": "pip install -e .[dev]",
    "log_parser": "parse_log_pytest", ...}
}
```

Figure 15: Raw instance record for 12rambau__sepal_ui-814.

Harbor task. Data preparation converts each raw instance into an executable Harbor task: a self-contained directory whose manifest declares the environment image and per-stage resource and timeout budgets, and whose verifier reproduces the official SWE-bench grading inside the sandbox.

```
12rambau__sepal_ui-814/
|-- task.toml           # manifest: image, resources, per-stage timeouts
|-- instruction.md      # the issue text shown to the agent
|-- environment/Dockerfile # FROM <task image>; WORKDIR /testbed; ...
'-- tests/
    |-- test.sh         # verifier: reset tests -> apply test_patch ->
    |                   # run -> grade (FAIL_TO_PASS pass, no regression)
    |-- test.patch      # the held-out test patch
    |-- parser.py       # output parser
    '-- config.json     # the raw instance record above

# task.toml (excerpt)
[environment]
docker_image = "sweb.eval.x86_64.12rambau_1776_sepal_ui-814:latest"
cpus = 1
memory_mb = 4096
build_timeout_sec = 1800.0
[agent]
timeout_sec = 3000.0
[verifier]
timeout_sec = 3000.0
```

Figure 16: The same instance as an executable Harbor task: directory layout and a manifest excerpt.

Agent-visible vs. verifier-only state. The directory layout in Figure 16 is host-side task state, not the agent’s filesystem. The agent phase (environment setup, agent execution) sees only the initialized repository. The `tests/` directory—containing `test.patch`, `config.json` (with the gold patch and `FAIL_TO_PASS`/`PASS_TO_PASS` lists), and grading scripts—is uploaded into the sandbox only during the verifier phase, after the agent has finished. This implements the “tests withheld until grading” defense described in Table 6.

Task-index row. The trainer samples from a thin task index; each row only points at a Harbor task, so the task content stays on shared storage and the index stays cheap to filter and re-mix.

```
{ "prompt": [{"role": "user", "content": "<path>/openswe_filtered/..."}],  
  "reward_model": {"style": "rule", "ground_truth": null},  
  "extra_info": {"data_source": "harbor",  
                 "harbor_task_path": "<path>/openswe_filtered/...",  
                 "instance_id": "4Catalyzer__flask-annex-37"} }
```

Figure 17: A task-index row: a pointer to a Harbor task rather than the task itself.